
FAST-AR: Fast Autoregressive Video Diffusion and World Models with Temporal Cache Compression and Sparse Attention

Dvir Samuel^{1 2} Issar Tzachor¹ Matan Levy³ Michael Green¹ Gal Chechik^{2 4} Rami Ben-Ari¹

Abstract

Autoregressive video diffusion models enable *streaming* generation, opening the door to long-form synthesis, video world models, and interactive neural game engines. However, their core attention layers become a major bottleneck at inference time: as generation progresses, the KV cache grows, causing both increasing latency and escalating GPU memory, which in turn restricts usable temporal context and harms long-range consistency. In this work, we study redundancy in autoregressive video diffusion and identify three persistent sources: near-duplicate cached keys across frames, slowly evolving (largely semantic) queries/keys that make many attention computations redundant, and cross-attention over long prompts where only a small subset of tokens matters per frame. Building on these observations, we propose a unified, training-free attention framework (**FAST-AR**) for **FAST-AutoRegressive** diffusion, consisting of three components: **TempCache** compresses the KV cache via temporal correspondence to bound cache growth; **AnnCA** accelerates cross-attention by selecting frame-relevant prompt tokens using fast approximate nearest neighbor (ANN) matching; and **AnnSA** sparsifies self-attention by restricting each query to semantically matched keys, also using a lightweight ANN. Together, these modules reduce attention, compute, and memory and are compatible with existing autoregressive diffusion backbones and world models. Experiments demonstrate up to $\times 5$ – $\times 10$ end-to-end speedups while preserving near-identical visual quality and, crucially, maintaining stable throughput and nearly constant peak GPU memory usage

over long rollouts, where prior methods progressively slow down and suffer from increasing memory usage. [Project Page](#)

1. Introduction

Video diffusion models (HaCohen et al., 2024; Wang et al., 2025; Kong et al., 2024) have achieved strong results in offline video synthesis, where all frames in a short clip are generated jointly, producing high visual fidelity and temporally coherent motion. Recently, *autoregressive* video diffusion models (Yin et al., 2025; Huang et al., 2025) have emerged to enable streaming generation: frames are produced sequentially in an online manner and can be consumed immediately. This transition from offline to online generation unlocks new applications, including long-form video generation (Liu et al., 2025b), controlled video world models (Agarwal et al., 2025; Gao et al., 2025), and neural game engines (Gao et al., 2025; HunyuanWorld, 2025).

Despite these new capabilities, autoregressive video diffusion models expose a critical bottleneck in their attention mechanisms, leading to two fundamental challenges: **(a) Generation speed.** 3D spatio-temporal attention scales with the number of cached keys. As the KV cache grows with each generated frame, the per-step attention cost (and thus latency) increases over time, making long or unbounded generation progressively slower. **(b) Memory efficiency.** The expanding KV cache also creates substantial memory overhead, limiting how much temporal context can fit on GPU. This often forces short context windows (Liu et al., 2025b), which in turn harms long-range temporal consistency.

Despite KV-cache efficiency being central to autoregressive video generation, it remains relatively underexplored: KV-cache compression techniques from NLP (for LLMs) can not be applied directly (Anonymous, 2026), and recent methods (Liu et al., 2025a; Anonymous, 2026) offer only modest speedups, sometimes with quality degradation. In parallel, most progress on accelerating 3D spatio-temporal attention targets *offline* video diffusion (Xi et al., 2025; Yang et al., 2025; Li* et al., 2025). Yet these gains do not reliably transfer to online autoregressive models, where the same methods frequently degrade quality and deliver limited, or even

¹OriginAI, Tel-Aviv, Israel ²Bar-Ilan University, Ramat-Gan, Israel ³The Hebrew University of Jerusalem, Jerusalem, Israel ⁴NVIDIA, Tel-Aviv, Israel. Correspondence to: Dvir Samuel <dvirsamuel@gmail.com>.

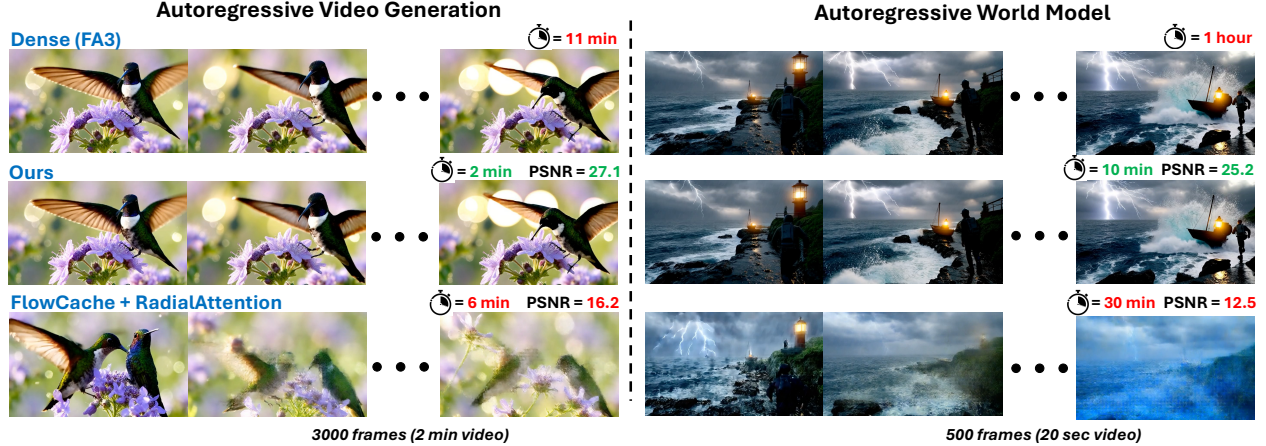


Figure 1. Our method substantially accelerates *pre-trained* autoregressive video diffusion models and autoregressive world models while maintaining high visual quality, by introducing a new KV-cache compression with self- and cross-attention sparsification. On a single H100 GPU, it achieves $5\times$ – $10\times$ speedups for multi-minute video generation, without further training/optimization, and keeps peak GPU memory nearly constant over long rollouts.

negative, speedups due to unfavorable memory and cache behavior. Following this, we therefore analyze whether autoregressive video diffusion contains exploitable redundancy, and find three consistent sources that are not directly addressed by existing sparse attention designs: (i) many cached keys are near-duplicates across frames, enabling aggressive KV merging; (ii) Q and K evolve slowly and are largely semantic, making many query–key dot products (attention score computations) redundant; and (iii) cross-attention wastes compute on long prompts although only a few tokens matter per frame.

Motivated by these observations, we introduce a unified attention framework, **FAST-AR** (**F**ast **A**uto**R**egressive diffusion), consisting of three components: **TempCache** compresses the KV cache using temporal correspondence; **An-nCA** prunes prompt tokens per frame, in cross-attention layers, using fast Approximate Nearest Neighbors (ANN) matching; and **AnnSA** sparsifies self-attention by restricting each query to semantically matched keys, also using lightweight ANN. To the best of our knowledge, this is the first use of ANN-based attention in autoregressive diffusion models in a fully training-free manner, requiring no retraining or fine-tuning. Experiments on autoregressive video diffusion and video world models show that our approach yields up to $\times 5$ – $\times 10$ end-to-end speedup (Figure 1), while maintaining nearly constant GPU memory over long rollouts, whereas baselines accumulate cache with increasing memory, and degrade video quality.

2. Related Work

Autoregressive video diffusion models and video world models. Modern video diffusion (Wang et al., 2025; Hun-

yuanWorld, 2025; HaCohen et al., 2024) models often adopt diffusion transformers (DiTs), which scale well but make video generation expensive due to long spatiotemporal token sequences and repeated denoising steps. To extend diffusion to long horizons, recent work (Yin et al., 2025; Huang et al., 2025) studies *autoregressive* (chunked) video diffusion, repeatedly denoising the next segment conditioned on previously generated history. SELF FORCING (Huang et al., 2025) reduces train–test mismatch by rolling out autoregressive generations with KV caching during training, optimizing predictions on self-generated context with a video-level objective. ROLLING FORCING (Liu et al., 2025b) targets multi-minute streaming by relaxing strict causality using joint denoising with increasing noise levels and by retaining early KV states as an attention sink to reduce error accumulation. In parallel, LONGVIE2 (Gao et al., 2025) frames long video generation as world modeling and proposes a staged training recipe to improve controllability and long-term consistency. Complementary to generation, DIFF-TRACK (Nam et al., 2025) studies temporal correspondence signals in video DiTs, showing that query–key similarity at specific layers/timesteps enables matching across frames for zero-shot point tracking.

KV compression and caching for autoregressive video diffusion. Since diffusion inference evaluates the denoiser repeatedly across timesteps, training-free caching can reuse intermediate computations when changes are small. TEA-CACHE (Liu et al., 2025a) uses a lightweight timestep-aware estimator to decide when cached outputs can be reused, providing speedups with limited quality loss. For long-horizon autoregressive generation, FLOWCACHE (Anonymous, 2026) proposes chunkwise recomputation policies and importance-based KV cache compression to bound

memory while accelerating ultra-long rollouts. Unlike (Liu et al., 2025a) and (Anonymous, 2026), we directly compress the *attention* KV cache by leveraging temporal correspondence across frames, merging redundant keys that track the same content over time. This training-free, correspondence-driven merging bounds KV growth and stabilizes both latency and peak memory over long rollouts.

Self-attention sparsification for diffusion models. A major bottleneck in video diffusion transformers is the quadratic cost of full 3D self-attention over space-time tokens. SVG (Xi et al., 2025) accelerates *offline* video diffusion by exploiting structured head sparsity, while SVG2 (Yang et al., 2025) improves token selection using semantic clustering and token permutation to reduce sparse-kernel inefficiency. RADIAL ATTENTION (Li* et al., 2025) uses an energy-decay prior to build static spatiotemporal masks with sub-quadratic cost. These methods target full-video generation; in contrast, self-attention sparsification tailored to autoregressive/chunked diffusion, with causality, KV caching, and long rollouts, remains largely unexplored.

Approximate nearest neighbor search. Approximate nearest neighbor (ANN) search accelerates high-dimensional similarity retrieval by trading exactness for speed, often using an offline preprocessing phase, e.g., graph indices such as HNSW (Malkov & Yashunin, 2016), hashing-based methods, or clustering-based indices (e.g., inverted files). Locality-sensitive hashing (LSH) (Indyk & Motwani, 1998) offers a lightweight alternative with principled hash families and multi-table bucket probing, avoiding exhaustive search while providing approximation guarantees. Quantization-based ANN (Jégou et al., 2011) compresses vectors into compact codes; product quantization (PQ) quantizes low-dimensional subspaces and enables fast distance estimation using lookup tables with asymmetric distance computation. Reformer (Kitaev et al., 2020) integrates LSH into the Transformer and trains around LSH-based attention routing. In this paper, we use LSH/quantization *training-free at inference time* to approximate nearest-neighbor matching without modifying or retraining the underlying model.

3. Auto-regressive Video Diffusion Models

We begin by reviewing the attention formulation used in autoregressive video diffusion models. Given queries $Q \in \mathbb{R}^{N_q \times d}$, keys $K \in \mathbb{R}^{N_k \times d}$, and values $V \in \mathbb{R}^{N_v \times d_v}$, attention is computed as

$$O = \text{softmax}\left(\frac{Q\hat{K}^\top}{\sqrt{d}}\right)\hat{V}, \quad (1)$$

where $\hat{K}$ and $\hat{V}$ denote the KV cache, formed by concatenating keys and values from the current frame with those from all previously generated frames. In cross-attention layers, $\hat{K}$

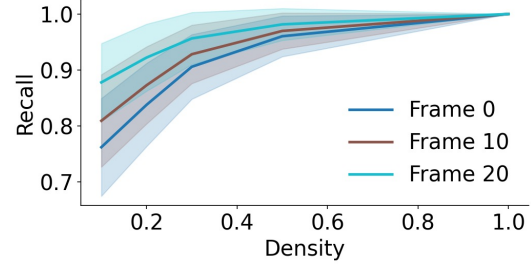


Figure 2. **Attention sparsity in autoregressive video diffusion.** Attention recall vs. density on Rolling-Forcing (Liu et al., 2025b), averaged over transformer blocks (shaded: std). Density is induced by keeping only the highest-attention entries. This achieves high recall, e.g., $\approx 85\%$ at 30% density, indicating substantial sparsity.

and $\hat{V}$ correspond to projected prompt tokens of the current frame. As generation proceeds, the KV cache grows with the number of generated frames, so the per-step attention cost scales linearly with cache length (and the cumulative attention work over a T -frame rollout can grow as $\mathcal{O}(T^2)$), leading to increasing inference latency and memory usage.

4. Motivation

Does attention in autoregressive video diffusion exhibit sparsity? Prior work (Yang et al., 2025) has shown that self-attention in *offline* video diffusion models is naturally sparse: only a small fraction of attention computations substantially contribute to the final output. However, it remains unclear whether similar sparsity exists in the *autoregressive* setting, where frames are generated sequentially and the attention context grows over time.

To investigate this, we generate 100 videos with Rolling-Forcing (Liu et al., 2025b) and quantify attention sparsity via *attention recall*, defined as the fraction of dense attention mass preserved after retaining only a top fraction of the largest attention entries. Specifically, for each transformer block, we keep the highest-attention entries, compute recall relative to dense attention, and report the mean and standard deviation across blocks. As shown in Figure 2, attention in autoregressive video diffusion models is highly sparse: retaining only 30% of the computations already preserves more than 85% of the attention mass, indicating substantial headroom for improving efficiency.

Where does sparsity arise during generation? To understand the source of this sparsity, we analyze the internal representations of attention modules across different generation timesteps. Our objective is to identify redundant computations that can be reduced or removed without degrading generation quality.

We first focus on the self-attention layers and examine the query (Q) and key ($\hat{K}$) features throughout generation. Fig-

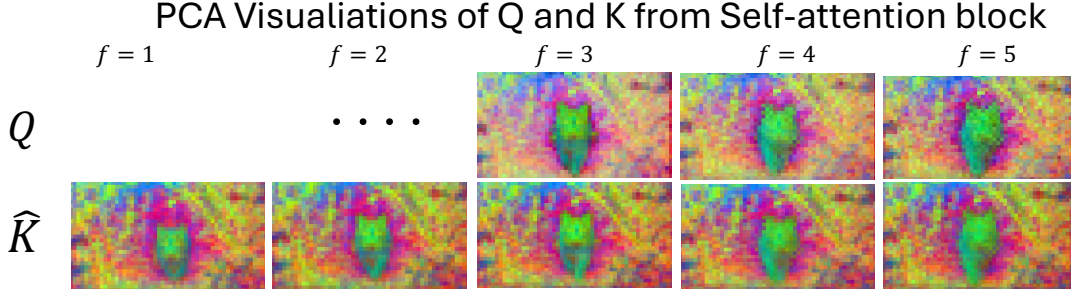


Figure 3. **Semantic structure and temporal redundancy in self-attention features.** PCA of self-attention queries Q and cached keys $\hat{K}$ across frames (similar colors denote nearby embeddings) for a generated video of a cat walking toward the camera. The features exhibit semantic clustering (foreground vs. background) and strong key repetition across frames, motivating KV-cache compression.

Figure 3 illustrates Principal Component Analysis (PCA) applied to Q and K features. Tokens with similar colors correspond to features that are close in the original embedding space. The visualization reveals two key properties. First, both Q and K are predominantly semantic: foreground tokens (e.g., the cat) cluster together and attend to corresponding semantic regions, while background tokens form separate clusters. Second, a large fraction of key features repeat across frames, with many keys from earlier frames reappearing later. This repetition persists across diffusion timesteps and in almost all transformer layers, suggesting that aggressive KV-cache compression is feasible.

We next examine the cross-attention layers by analyzing the attention maps between textual prompt tokens and video latents. As depicted in Figure 4, autoregressive video diffusion models are typically conditioned on long, detailed prompts (Huang et al., 2025; Liu et al., 2025b) that describe objects, actions, and events spanning the entire video. As a result, cross-attention incurs substantial computational overhead: at every layer and generation timestep, each query attends to all prompt tokens, even though only a small subset is relevant for synthesizing the current frame. Cross-attention is highly selective: for each frame, most attention mass concentrates on a small subset of prompt tokens, while the rest contribute little. This observation suggests that dynamically selecting a frame-specific subset of relevant prompt tokens can dramatically reduce cross-attention computation without sacrificing quality.

5. Method

Our goal is to accelerate autoregressive video diffusion by reducing the compute and memory cost of attention without degrading generation quality. We do so with three components: (i) **TempCache** for temporal KV-cache compression, (ii) **AnnSA** for sparse self-attention, and (iii) **AnnCA** for sparse cross-attention; all three use fast approximate nearest-neighbor matching to select a small set of candidate tokens before computing attention.

5.1. Attention as Approximate Nearest Neighbor Search

Directly computing attention scores via $Q\hat{K}^\top$ is expensive. Efficient kernels such as FlashAttention (Shah et al., 2024) and FlashInfer (Ye et al., 2025) avoid explicitly materializing this matrix, but the overall cost still scales with the number of cached keys per step, and thus becomes prohibitive over long autoregressive generations.

Motivated by our empirical findings (Section 4), which show that only a small fraction of keys meaningfully contribute to each query, we reinterpret attention as an *approximate nearest neighbor* (ANN) problem. For each query, the goal is to identify the keys with the largest dot products and compute attention only over them.

Formally, instead of attending to all cached keys $\hat{K}$ (and values $\hat{V}$), we first retrieve a small candidate set of key indices

$$\mathcal{N}(q) \subseteq \{1, \dots, |\hat{K}|\},$$

and denote by $\hat{K}_{\mathcal{N}(q)} \in \mathbb{R}^{|\mathcal{N}(q)| \times d}$ and $\hat{V}_{\mathcal{N}(q)} \in \mathbb{R}^{|\mathcal{N}(q)| \times d_v}$ the corresponding rows of the KV cache. We then approximate attention as

$$\text{Attn}(q) \approx \text{Softmax}\left(\frac{q\hat{K}_{\mathcal{N}(q)}^\top}{\sqrt{d}}\right) \hat{V}_{\mathcal{N}(q)}.$$

Since ANN search must be performed repeatedly across layers and diffusion timesteps, it must incur minimal preprocessing overhead. We consider two lightweight approximate nearest neighbor strategies:

Locality-Sensitive Hashing (LSH). We project queries and keys into a shared low-dimensional random subspace and bucket them with locality-sensitive hashing (Indyk & Motwani, 1998), so that a query only compares against keys that land in the same bucket(s).

Quantized Similarity Search. Alternatively, we quantize (Jégou et al., 2011) queries and keys to a low-bit representation and perform nearest-neighbor search directly in the quantized space.

"A white-and-orange tabby cat walks toward the camera along a narrow garden path, filmed from a ground-level, cinematic angle with warm tones and dappled sunlight. A van passes through the frame, briefly blocking the view; when it clears, the cat has transformed into a golden retriever continuing forward at the same pace. The dog looks calm and happy, its golden fur glowing against a softly blurred, leafy background.."



Figure 4. **Cross-attention is frame-selective.** Input prompt (top) with Per-token cross-attention maps (bottom) for “cat”, “van”, and “dog” across frames. Attention concentrates on the tokens relevant to the current content (cat early, van during occlusion, dog after transformation), suggesting that pruning irrelevant prompt tokens per frame can reduce cross-attention compute.

Both approaches reduce matching cost by limiting dot products to a small candidate set, lowering compute and memory bandwidth while retaining sufficient attention recall, and do so without offline preprocessing, enabling fast inference-time matching. After candidates are selected, attention is computed with sparse attention kernels (Ye et al., 2025).

Having established a fast approximation for attention computation, we next focus on reducing the size of the KV cache itself.

5.2. KV Cache Compression using Temporal Correspondence

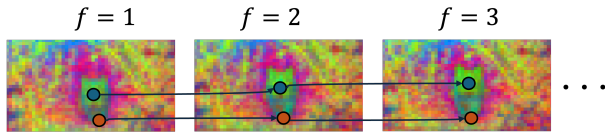


Figure 5. **Temporal correspondence for KV compression.** Many key features persist across frames and can be matched using temporal correspondence (colored dots/arrows). Correspondences are recovered by selecting, for each current-frame query, the most related key in a previous frame (Nam et al., 2025)

In Section 4, we showed that key features exhibit substantial temporal redundancy: many keys from earlier frames reappear nearly unchanged in later frames. This observation suggests that the KV cache can be dramatically compressed by identifying temporally corresponding tokens and retaining only representative keys. Figure 5 illustrates this phenomenon: tokens corresponding to the same semantic regions persist across frames and follow consistent trajectories in feature space.

Finding temporal correspondences. To identify corresponding tokens across frames, we build on the observation from DiffTrack (Nam et al., 2025) that temporal correspondence can be recovered directly from attention. Specifically, for a query token in the current frame, its corresponding token in a previous frame is the key with the highest attention score. To avoid explicitly computing $QK^\top$, we instead use our ANN machinery (section 5.1) to retrieve the top-1 nearest neighbor key for each query. This provides an efficient estimate of temporal correspondence during generation.

Once correspondences are established, we group keys across frames that correspond to the same semantic token. For each group, we retain only the recent representative key in the KV cache. However, naively removing keys raises three concerns: (1) How to apply this compression without explicitly forming the attention score matrix $QK^\top$, since current kernels do not materialize it, (2) how to correctly aggregate values $\hat{V}$, and (3) whether attention computed on the compressed cache remains accurate. We address all concerns with the following lemma, which shows that attention over duplicate keys can be computed *exactly* using a grouped representation.

Lemma 5.1 (Redundancy-free attention). *Let $q \in \mathbb{R}^{d_k}$ be a query, and let $K = (k_1, \dots, k_n)^\top$ and $V = (v_1, \dots, v_n)^\top$ denote keys and values. Suppose the indices are partitioned into groups $\{G_t\}_{t=1}^g$ such that all keys within a group are identical. Then the standard attention over (K, V) is exactly equal to an attention computed over the g group representatives: for group t , the logit is shifted by $+\log m_t$, where $m_t = |G_t|$, and the associated value is the mean of $\{v_i\}_{i \in G_t}$.*

The full statement and proof are given in the appendix.

In other words, the lemma states that merging duplicate keys incurs *no* approximation error: when redundancy is exact, attention can be computed exactly from a compressed KV cache. In practice, temporal features are only approximately redundant, so keys are merged using a similarity threshold, yielding a controlled approximation. In the worst case, when no redundancy is detected, the method reduces to standard dense attention. Importantly, the procedure is kernel-agnostic: it modifies only the *inputs* to attention (the cached K, V) and requires no changes to the attention kernel itself. We refer to this approach as **TempCache** (Temporal correspondence KV Cache compression).

5.3. Cross-Attention Redundancy

Autoregressive video diffusion models are often conditioned on long textual prompts (Figure 4), making cross-attention expensive: at each layer and timestep, queries attend to all prompt tokens, although only a few are relevant for the current frame. We reduce this overhead by selecting only the prompt tokens that are relevant to the current frame. Crucially, we avoid computing dense attention maps between all latent queries and all prompt keys. Instead, we leverage approximate nearest neighbor (ANN) search to identify relevant prompt tokens efficiently. Concretely, we project both the latent queries of the current frame and the prompt keys into a shared LSH- or quantized embedding space. For each prompt token, we check whether it has at least one neighboring latent query in its bucket. Prompt tokens that do not share any bucket with any current-frame query are considered irrelevant and excluded from cross-attention computation for that frame. We call this approach **AnnCA** (Approximate Nearest-Neighbor Cross-Attention).

5.4. Self-Attention Redundancy

Self-attention in video diffusion models exhibits strong structural patterns: tokens tend to attend primarily to other tokens that are semantically related and spatially or temporally proximate (Figure 3). We exploit this structure to sparsify self-attention by reusing the semantic buckets discovered during cross-attention pruning (Section 5.3), effectively transferring prompt-induced semantic grouping into self-attention. Each token’s query is assigned to one or more buckets, and during self-attention, we restrict it to attend only to keys within the same bucket(s), enforcing semantic locality. We compute this bucketed attention efficiently using block-sparse attention kernels (e.g., FlashInfer (Ye et al., 2025)). We call this approach **AnnSA** (Approximate Nearest Neighbor Self Attention).

6. Experiments

6.1. Setup

Models. We evaluate on state-of-the-art open-source autoregressive video diffusion and world models: (i) **Rolling-Forcing** (Liu et al., 2025b), a real-time long-horizon autoregressive video diffusion method for multi-minute generation and (ii) **LongVie2** (Gao et al., 2025), an autoregressive transformer-based world model that frames long video generation as world modeling and introduces a staged training recipe (multimodal guidance and history alignment) to improve controllability and long-term consistency.

Metrics. Following prior work on efficient video diffusion attention (Xi et al., 2025; Yang et al., 2025), we evaluate fidelity to the dense-attention baseline using PSNR, SSIM, and LPIPS computed between videos generated with our approach and videos generated with Dense FlashAttention-3 (FA3) under matched prompts and random seeds. We additionally report LongVBench (Yang et al., 2025)/LongVGenBench (Gao et al., 2025) scores as measures of perceptual video quality. To quantify attention efficiency, we report the attention density (fraction of executed query-key interactions relative to dense attention) and attention recall (fraction of attention mass preserved by the sparse pattern). Finally, we report per-component and end-to-end *generation acceleration* as the speedup over Dense FA3.

Datasets. Consistent with (Xi et al., 2025; Yang et al., 2025), we evaluate on **LongVBench**, a long-horizon benchmark based on VBench prompts (Penguin Benchmark with the released prompt optimization), designed to stress long-context generation. We also evaluate on **LongVGenBench** (Gao et al., 2025), a long-video generation benchmark that measure long-range quality under extended generation.

Baselines. We compare our approach with: (i) **Dense attention**, implemented using FlashAttention-3 (Shah et al., 2024) as our primary exact-attention baseline; (ii) **KV-cache/computation reuse methods**, including TeaCache (Liu et al., 2025a), a training-free timestep-aware caching strategy that reuses intermediate computations when consecutive denoising steps are similar, and FlowCache (Anonymous, 2026), a caching framework tailored to autoregressive video generation with chunk-wise policies and importance-based cache control; and (iii) **Sparse attention methods**, including SVG (Xi et al., 2025), SVG2 (Yang et al., 2025), and RadialAttn (Li* et al., 2025), which we adapt and implement for autoregressive video diffusion models to enable a direct comparison in the streaming setting.

For all baselines, we perform a full grid search over the reported hyperparameters to obtain the best performance in our setting. In the supplementary material, we further report results on the exact same datasets, settings, and evaluation tables used in the corresponding baseline papers (for both

Table 1. **Rolling-Forcing (LongVBench)**. Our method achieves the best quality–efficiency trade-off across all settings. TempCache (LSH/Quant) compresses the KV cache with the highest recall while matching dense attention quality. Combining TempCache with ANN-based SA/CA yields up to $\times 10.7$ – $\times 10.8$ end-to-end speedup.

RollingForcing	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	VBench $\uparrow$	Min Density $\downarrow$	Max Recall $\uparrow$	Component Speedup $\uparrow$	E2E Speedup $\uparrow$
	Dense (FlashAttention 3)	–	–	–	84.08	100%	100%	$\times 1.0$	$\times 1.0$
KV compression									
	TeaCache	16.12	0.315	0.523	84.11	93.2%	84.6%	$\times 1.1$	–
	FlowCache	22.15	0.634	0.222	84.15	82.9%	86.9%	$\times 2.3$	–
	TempCache-LSH (ours)	24.13	0.651	0.149	84.17	16.8%	90.2%	$\times 6.8$	$\times 3.0$
	TempCache-Quant (ours)	24.26	0.653	0.143	84.19	16.2%	91.4%	$\times 6.9$	$\times 3.2$
Sparse Self-Attention									
	SVG1	14.22	0.201	0.744	33.15	76.1%	30.2%	$\times 0.1$	–
	SVG2	14.29	0.226	0.736	34.66	68.8%	38.9%	$\times 0.2$	–
	RadialAttn	16.87	0.289	0.702	61.51	81.6%	58.3%	$\times 2.8$	–
	AnnSA-LSH (ours)	25.73	0.688	0.142	83.25	27.6%	92.4%	$\times 5.1$	$\times 2.7$
	AnnSA-Quant (ours)	25.77	0.689	0.141	83.29	28.0%	92.6%	$\times 5.2$	$\times 2.8$
Sparse Cross-Attention									
	AnnCA-LSH (ours)	25.68	0.679	0.155	83.23	33.1%	94.2%	$\times 2.2$	$\times 1.2$
	AnnCA-Quant (ours)	24.11	0.646	0.148	82.89	29.5%	91.1%	$\times 2.3$	$\times 1.2$
Full									
	FlowCache+RadialAttn	16.98	0.294	0.687	45.15	–	–	–	$\times 4.4$
	All Ours-LSH	25.71	0.681	0.147	84.02	–	–	–	$\times 10.7$
	All Ours-Quant	25.73	0.678	0.147	83.99	–	–	–	$\times 10.8$

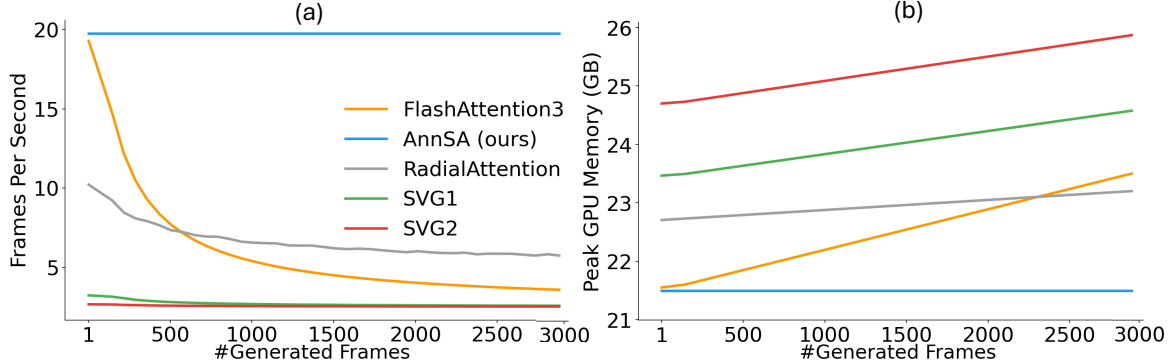


Figure 6. **Scaling with generation length in Rolling-Forcing**. (a) *Throughput*. As context grows, Dense FA3 slows substantially, and prior sparsification baselines (SVG1/2, RadialAttention) fail to maintain throughput due to heavy per-block preprocessing repeated across blocks, timesteps, and frames. Our method sustains nearly constant FPS over a 3K-frame rollout, keeping attention cost effectively independent of cache length. (b) *Peak memory*. FA3 and baselines exhibit increasing GPU memory as the KV cache expands, while our memory remains flat, consistent with a bounded cache.

short video generation and KV-cache compression).

Implementation details. All experiments are run on a single H100 GPU. To measure long-horizon scaling, we generate streams of 3000 frames *without* imposing a context-window bound on the KV cache for any method. Following prior work (Xi et al., 2025), sparse attention is disabled for the first 30% of denoising steps, and sparsification is applied only to blocks that empirically exhibit sparsity (about 70% of blocks). We use FAISS (Douze et al., 2025) for LSH and quantization-based ANN retrieval, and FlashInfer (Ye et al., 2025) kernels for sparse attention. Unless stated otherwise, quantization uses 8-bit (ablations in Supp.). Qualitative

figures show one representative frame per video.

7. Results

7.1. Quantitative Results

Table 1 compares our method with Dense FlashAttention-3 and current SoTA baselines on Rolling-Forcing LongVBench. It shows that our approach achieves the best quality–efficiency trade-off across all settings: both **TempCache-LSH/Quant** compress the KV cache aggressively (down to $\sim 16\%$ Min Density) while preserving high attention recall (~ 90 – 91%) and matching dense quality

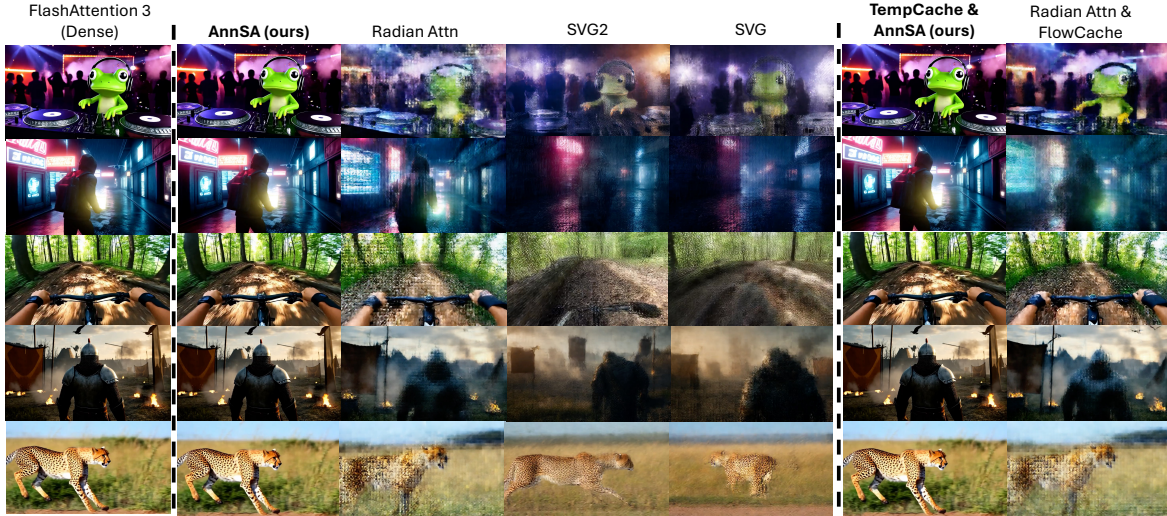


Figure 7. **Qualitative results on long-video generation with Rolling-Forcing.** Our approach preserves the visual fidelity and temporal consistency of Dense FA3 across diverse prompts, while current sparsification baselines (SVG1/2) often exhibit artifacts and drift; RadialAttention is more stable but still degrades in challenging scenes.

(VBench ≈ 84.1). In contrast, TeaCache and FlowCache provide only modest speedups ($\times 1.1$ – $\times 1.3$) while retaining most of the dense computation. For attention sparsification, offline-designed baselines (SVG1/2, RadialAttn) substantially degrade quality and/or recall, whereas our ANN-based self- and cross-attention pruning maintains near-dense quality at low density with consistently high recall. Finally, combining cache compression with SA/CA sparsity yields the strongest overall gains: **All Ours-LSH/Quant** achieves up to $\times 10.7$ – $\times 10.8$ end-to-end speedup while preserving FA3-level quality, while the best baseline combination (FlowCache+RadialAttn) reaches only $\times 4.4$ and collapses in quality. Results for generative world-model (LongVie2) in Supp.

Figure 6 shows how Rolling-Forcing (Liu et al., 2025b) scales with generation length. **(a) Throughput.** As the KV cache grows, the dense attention baseline (FA3) slows down sharply, with frame-per-second (FPS) dropping continuously over the 3K-frame rollout. Existing sparsification baselines (SVG1/2, RadialAttention) also fail to sustain throughput in this long-horizon regime, largely due to substantial per-block preprocessing (e.g., clustering or energy/decay estimation) that must be repeated across transformer blocks, diffusion timesteps, and frames. In contrast, our method maintains constant FPS throughout generation: SA sparsity prevents attention cost from increasing with context length, and our ANN matching remains lightweight and kernel-friendly at scale. **(b) Peak memory.** Peak GPU memory for the dense attention baseline and current approaches increases with the expanding KV cache, whereas our method stays constant. The same trend is also observed for world models; results on LongVie2 are provided in the supplementary material.

7.2. Qualitative Results

Figure 7 provides qualitative comparisons on long video generation with Rolling-Forcing. Across diverse prompts (characters, landscapes, fast motion, and complex textures), our method preserves the key visual attributes of Dense FlashAttention-3, including subject identity, fine details, and coherent motion, while sustaining stable appearance over time. In contrast, baseline sparse attention methods adapted from offline diffusion (e.g., SVG1/2) frequently introduce severe artifacts and temporal drift, leading to blurred structure, washed-out textures, or identity collapse as generation progresses. RadialAttention is more stable than SVG-style sparsity but still shows noticeable degradation relative to dense attention in challenging scenes. More qualitative results can be found in the appendix.

8. Summary

In this paper, we study the attention in autoregressive video diffusion, where KV-cache growth makes streaming generation slower and more memory-intensive over time. We identify three redundancies: duplicate keys across frames, slowly evolving semantic Q/K, and expensive cross-attention over long prompts. We introduce a unified, training-free framework: **TempCache** (temporal KV merging), **AnnCA** (ANN-based prompt pruning), and **AnnSA** (ANN-based sparse self-attention). Our method is plug-and-play and requires no retraining or fine-tuning. Experiments show up to $\times 5$ – $\times 10$ end-to-end speedups with constant GPU memory over long video generations while preserving video quality.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Agarwal, N. N., Ali, A., Bala, M., Balaji, Y., Barker, E., Cai, T., Chattopadhyay, P., Chen, Y., Cui, Y., Ding, Y., Dworakowski, D., Fan, J., Fenzi, M., Ferroni, F., Fidler, S., Fox, D., Ge, S., Ge, Y., Gu, J., Gururani, S., He, E., Huang, J., Huffman, J. S., Jannaty, P., Jin, J., Kim, S. W., Kl'ar, G., Lam, G., Lan, S., Leal-Taixé, L., Li, A., Li, Z., Lin, C.-H., Lin, T.-Y., Ling, H., Liu, M.-Y., Liu, X., Luo, A., Ma, Q., Mao, H., Mo, K., Mousavian, A., Nah, S., Niverty, S., Page, D., Paschalidou, D., Patel, Z., Pavao, L., Ramezanali, M., Reda, F. A., Ren, X.-S., Sabavat, V. R. N., Schmerling, E., Shi, S., Stefaniak, B., Tang, S., Tchapmi, L. P., Tredak, P., Tseng, W.-C., Varghese, J. R., Wang, H., Wang, H., Wang, H., Wang, T., Wei, F., Wei, X., Wu, J. Z., Xu, J., Yang, W., Yen-Chen, L., Zeng, X., Zeng, Y., Zhang, J., Zhang, Q., Zhang, Y., Zhao, Q., and Zolkowski, A. Cosmos world foundation model platform for physical ai. *ArXiv*, 2025.
- Anonymous. Flow caching for autoregressive video generation. In *ICLR 2026 Conference Submission*, 2026. URL <https://openreview.net/forum?id=vko4DuhKbh>. OpenReview submission.
- Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library, 2025.
- Gao, J., Chen, Z., Liu, X., Zhuang, J., Xu, C., Feng, J., Qiao, Y., Fu, Y., Si, C., and Liu, Z. Longvie 2: Multimodal controllable ultra-long video world model, 2025. URL <https://arxiv.org/abs/2512.13604>.
- HaCohen, Y., Chiprut, N., Brazowski, B., Shalem, D., Moshe, D.-P., Richardson, E., Levin, E. I., Shiran, G., Zabari, N., Gordon, O., Panet, P., Weissbuch, S., Kulikov, V., Bitterman, Y., Melumian, Z., and Bibi, O. Ltx-video: Realtime video latent diffusion. 2024.
- Huang, X., Li, Z., He, G., Zhou, M., and Shechtman, E. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *NeurIPS*, 2025.
- HunyuanWorld, T. Hunyuanworld 1.0: Generating immersive, explorable, and interactive 3d worlds from words or pixels. *arXiv preprint*, 2025.
- Indyk, P. and Motwani, R. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, 1998.
- Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2011.
- Kitaev, N., Łukasz Kaiser, and Levskaya, A. Reformer: The efficient transformer, 2020.
- Kong, W., Tian, Q., Zhang, Z., Min, R., Dai, Z., Zhou, J., Xiong, J., Li, X., Wu, B., Zhang, J., et al. Hunyuan-video: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.
- Li*, X., Li*, M., Cai, T., Xi, H., Yang, S., Lin, Y., Zhang, L., Yang, S., Hu, J., Peng, K., Agrawala, M., Stoica, I., Keutzer, K., and Han, S. Radial attention: $\mathcal{O}(n \log n)$ sparse attention with energy decay for long video generation. *NeurIPS*, 2025.
- Liu, F., Zhang, S., Wang, X., Wei, Y., Qiu, H., Zhao, Y., Zhang, Y., Ye, Q., and Wan, F. Timestep embedding tells: It's time to cache for video diffusion model, 2025a.
- Liu, K., Hu, W., Xu, J., Shan, Y., and Lu, S. Rolling forcing: Autoregressive long video diffusion in real time. *arXiv preprint arXiv:2509.25161*, 2025b.
- Malkov, Y. and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2016.
- Nam, J., Son, S., Chung, D., Kim, J., Jin, S., Hur, J., and Kim, S. Emergent temporal correspondences from video diffusion transformers, 2025.
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024.
- Wang, A., Ai, B., Wen, B., Mao, C., Xie, C.-W., Chen, D., Yu, F., Zhao, H., Yang, J., Zeng, J., Wang, J., Zhang, J., Zhou, J., Wang, J., Chen, J., Zhu, K., Zhao, K., Yan, K., Huang, L., Meng, X., Zhang, N., Li, P., Wu, P., Chu, R., Feng, R., Zhang, S., Sun, S., Fang, T., Wang, T., Gui, T., Weng, T., Shen, T., Lin, W., Wang, W., Wang, W., Zhou, W.-C., Wang, W., Shen, W., Yu, W., Shi, X., Huang, X., Xu, X., Kou, Y., Lv, Y.-M., Li, Y., Liu, Y., Wang, Y., Zhang, Y., Huang, Y., Li, Y., Wu, Y., Liu, Y., Pan, Y., Zheng, Y., Hong, Y., Shi, Y., Feng, Y., Jiang, Z., Han, Z., Wu, Z., and Liu, Z. Wan: Open and advanced large-scale video generative models. *ArXiv*, 2025.
- Xi, H., Yang, S., Zhao, Y., Xu, C., Li, M., Li, X., Lin, Y., Cai, H., Zhang, J., Li, D., et al. Sparse videogen: Accelerating video diffusion transformers with spatial-temporal sparsity. *ICML*, 2025.

Yang, S., Xi, H., Zhao, Y., Li, M., Zhang, J., Cai, H., Lin, Y., Li, X., Xu, C., Peng, K., et al. Sparse videogen2: Accelerate video generation with sparse attention via semantic-aware permutation. *NeurIPS*, 2025.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005*, 2025.

Yin, T., Zhang, Q., Zhang, R., Freeman, W. T., Durand, F., Shechtman, E., and Huang, X. From slow bidirectional to fast autoregressive video diffusion models. In *CVPR*, 2025.

A. Proof of Lemma 5.1

Lemma A.1 (Redundancy-free attention). *Let $q \in \mathbb{R}^{d_k}$ be a query, and let $K = (k_1, \dots, k_n)^\top \in \mathbb{R}^{n \times d_k}$ and $V = (v_1, \dots, v_n)^\top \in \mathbb{R}^{n \times d_v}$ denote keys and values. Consider standard scaled dot-product attention*

$$\text{Attn}(q, K, V) = \frac{\sum_{i=1}^n e^{s_i} v_i}{\sum_{i=1}^n e^{s_i}}, \quad s_i = \frac{q^\top k_i}{\sqrt{d_k}}. \quad (2)$$

Assume the index set $\{1, \dots, n\}$ is partitioned into g disjoint groups $\{G_t\}_{t=1}^g$ such that all keys inside each group are identical, i.e.,

$$k_i = k'_t \quad \forall i \in G_t, \quad (3)$$

for some representative key $k'_t \in \mathbb{R}^{d_k}$. Define the group size (multiplicity) $m_t = |G_t|$ and the mean value

$$\tilde{v}_t = \frac{1}{m_t} \sum_{i \in G_t} v_i. \quad (4)$$

Let $s_t = \frac{q^\top k'_t}{\sqrt{d_k}}$ and $\tilde{s}_t = s_t + \log m_t$. Then

$$\text{Attn}(q, K, V) = \sum_{t=1}^g \frac{e^{\tilde{s}_t}}{\sum_{u=1}^g e^{\tilde{s}_u}} \tilde{v}_t. \quad (5)$$

Equivalently, attention over (q, K, V) is exactly equal to attention over the g representative keys $K' = (k'_1, \dots, k'_g)$ and mean values $\tilde{V} = (\tilde{v}_1, \dots, \tilde{v}_g)$, with an additive bias $\log m_t$ applied to the logits.

Proof. Because all keys within a group G_t are identical by (3), they induce the same attention score. For any $i \in G_t$,

$$s_i = \frac{q^\top k_i}{\sqrt{d_k}} = \frac{q^\top k'_t}{\sqrt{d_k}} = s_t. \quad (6)$$

We now regroup the numerator and denominator of (2) by groups.

Regrouping the numerator.

$$\sum_{i=1}^n e^{s_i} v_i = \sum_{t=1}^g \sum_{i \in G_t} e^{s_i} v_i = \sum_{t=1}^g \sum_{i \in G_t} e^{s_t} v_i = \sum_{t=1}^g e^{s_t} \sum_{i \in G_t} v_i. \quad (7)$$

Regrouping the denominator.

$$\sum_{i=1}^n e^{s_i} = \sum_{t=1}^g \sum_{i \in G_t} e^{s_i} = \sum_{t=1}^g \sum_{i \in G_t} e^{s_t} = \sum_{t=1}^g m_t e^{s_t}. \quad (8)$$

Substituting (7) and (8) into (2) yields

$$\text{Attn}(q, K, V) = \frac{\sum_{t=1}^g e^{s_t} \sum_{i \in G_t} v_i}{\sum_{t=1}^g m_t e^{s_t}}. \quad (9)$$

Next, observe that $m_t e^{s_t} = e^{s_t + \log m_t}$, and rewrite the numerator to expose the group mean values:

$$\begin{aligned} \text{Attn}(q, K, V) &= \frac{\sum_{t=1}^g e^{s_t} \sum_{i \in G_t} v_i}{\sum_{t=1}^g m_t e^{s_t}} = \frac{\sum_{t=1}^g (m_t e^{s_t}) \left(\frac{1}{m_t} \sum_{i \in G_t} v_i \right)}{\sum_{t=1}^g m_t e^{s_t}} \\ &= \frac{\sum_{t=1}^g e^{s_t + \log m_t} \tilde{v}_t}{\sum_{t=1}^g e^{s_t + \log m_t}}. \end{aligned} \quad (10)$$

Table 2. **LongVie2 (LongVGenBench)**. Quality–efficiency trade-off vs. dense FA3 across KV compression, sparse SA, sparse CA, and the full system. TempCache (LSH/Quant) compresses the KV cache to $\sim 33.1\%$ Min Density with high recall, ANN-based SA/CA preserve near-baseline quality at low density, and the full method achieves $\times 6.3$ – $\times 6.9$ speedup.

LongVie2	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	LongVGenBench $\uparrow$	Min Density $\downarrow$	Max Recall $\uparrow$	Component Speedup $\uparrow$
	FlashAttention 3	–	–	–	69.67	100%	100%	$\times 1.0$
KV compression								
	TeaCache	13.6	0.254	0.399	61.41	92.0%	85.3%	$\times 1.1$
	FlowCache	20.6	0.315	0.357	62.78	90.4%	87.6%	$\times 1.1$
	TempCache-LSH (ours)	23.0	0.523	0.243	63.10	33.4%	91.3%	$\times 3.5$
	TempCache-Quant (ours)	23.4	0.531	0.223	63.16	33.1%	92.4%	$\times 3.7$
Sparse SA								
	SVG1	11.6	0.557	0.831	24.83	78.3%	11.8%	$\times 0.3$
	SVG2	12.7	0.185	0.812	25.65	70.9%	15.9%	$\times 0.5$
	RadialAttn	17.2	0.197	0.644	41.11	82.4%	44.7%	$\times 2.7$
	AnnSA-LSH (ours)	23.9	0.601	0.222	65.32	40.3%	89.5%	$\times 5.3$
	AnnSA-Quant (ours)	23.9	0.633	0.199	66.43	44.9%	87.0%	$\times 5.4$
Sparse CA								
	AnnCA-LSH (ours)	24.6	0.634	0.197	66.67	42.8%	90.0%	$\times 1.7$
	AnnCA-Quant (ours)	23.3	0.620	0.203	63.77	44.9%	85.6%	$\times 1.8$
Full								
	FlowCache+RadialAttn	18.6	0.247	0.572	49.84	–	–	$\times 3.1$
	All Ours-LSH	23.5	0.611	0.216	64.91	–	–	$\times 6.3$
	All Ours-Quant	23.5	0.613	0.218	63.69	–	–	$\times 6.9$

Defining $\tilde{s}_t = s_t + \log m_t$ turns (10) into a softmax over groups:

$$\text{Attn}(q, K, V) = \sum_{t=1}^g \frac{e^{\tilde{s}_t}}{\sum_{u=1}^g e^{\tilde{s}_u}} \tilde{v}_t, \quad (11)$$

which is exactly (5). $\square$

Lemma A.1 implies that if multiple positions share identical keys, they can be merged without any approximation error by (i) averaging their values and (ii) adding a $\log m_t$ bias to the corresponding logit. When no duplicate keys exist, $m_t = 1$ for all t and the formula reduces to standard attention.

B. Additional Results

B.1. Quantitative Results

Table 2 reports quantitative results on LongVGenBench with the LongVie2 world model. It shows that our components consistently improve the quality–efficiency trade-off relative to dense FlashAttention-3 and prior baselines. For **KV compression**, TeaCache/FlowCache provide only $\times 1.1$ speedup while retaining $\sim 90\%$ density, whereas TempCache (LSH/Quant) reduces density to $\approx 33\%$ with high recall (91.3–92.4%) and improves LongVGenBench (63.10–63.16) at $\times 3.5$ – $\times 3.7$ speedup. For **sparse self-attention**, offline-designed sparsification (SVG1/2, RadialAttn) severely degrades LongVGenBench (24.83–41.11) and recall, while AnnSA (LSH/Quant) preserves much stronger quality (65.32–66.43) with high recall (87.0–89.5%) and achieves $\times 5.3$ – $\times 5.4$ speedup. For **sparse cross-attention**, AnnCA maintains high recall (85.6–90.0%) and strong quality (63.77–66.67) at $\times 1.7$ – $\times 1.8$ speedup. Finally, the **full system** delivers the best overall performance: All Ours achieves $\times 6.3$ – $\times 6.9$ speedup with substantially higher LongVGenBench (63.69–64.91) than the strongest baseline combination FlowCache+RadialAttn (49.84), indicating that our method scales effectively to world-model generation without sacrificing quality.

Figure 8 shows attention scaling on LongVie2 as generation length increases. (a) **Attention time**. Dense FlashAttention-3 exhibits steadily increasing attention latency with more frames, and RadialAttention remains slower than our method;

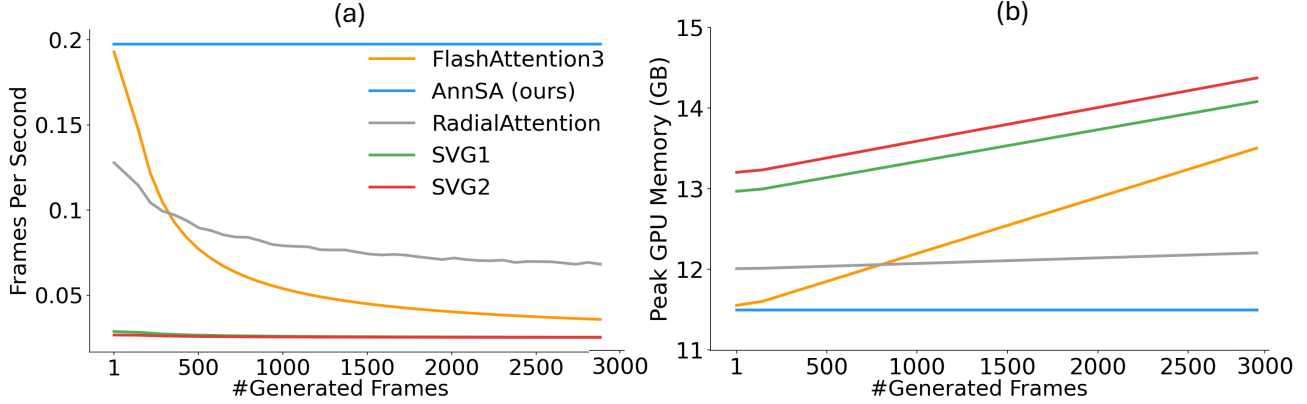


Figure 8. **Scaling with generation length in LonVie2.** (a) *Throughput*. As the context grows, dense FA3 slows down, and existing sparsification baselines (SVG1/2, RadialAttention) do not sustain throughput due to substantial per-block preprocessing repeated across transformer blocks, diffusion timesteps, and frames. Our method keeps FPS nearly constant over a 3K-frame rollout, making attention cost effectively independent of cache length. (b) *Peak memory*. FA3 and baselines show rising GPU memory with the expanding KV cache, whereas our memory stays flat, consistent with a bounded cache.

Table 3. **TempCache-Quant on MAGI-1 and SkyReels-V2.** Results follow exactly the evaluation protocol of (Anonymous, 2026). TempCache-Quant achieves the best efficiency–quality trade-off across both models, reducing PFLOPs and latency while maintaining or improving perceptual quality (VBench/LPIPS/SSIM/PSNR).

Model	Method	PFLOPs↓	Speedup↑	Latency (s)↓	VBench↑	LPIPS↓	SSIM↑	PSNR↑
MAGI-1	Vanilla	306	1×	2873	77.06%	–	–	–
	TeaCache-slow	294	1.12×	2579	77.50%	0.6211	0.2801	13.26
	TeaCache-fast	225	1.44×	1998	70.11%	0.8160	0.1138	8.94
	FlowCache-slow	161	1.86×	1546	78.96%	0.3160	0.6497	22.34
	FlowCache-fast	140	2.38×	1209	77.93%	0.4311	0.5140	19.27
	TempCache-Quant (ours)	110	4.11×	1009	78.99%	0.3156	0.6555	23.12
SkyReels-V2	Vanilla	113	1×	1540	83.84%	–	–	–
	TeaCache-slow	58	1.89×	814	82.67%	0.1472	0.7501	21.96
	TeaCache-fast	49	2.2×	686	80.06%	0.3063	0.6121	18.39
	FlowCache-slow	36	5.88×	262	83.12%	0.1225	0.7890	23.74
	FlowCache-fast	28	6.7×	230	83.05%	0.1467	0.7635	22.95
	TempCache-Quant (ours)	11	9.25×	146	83.82%	0.1160	0.8020	23.99

SVG1/2 are orders-of-magnitude slower (log-scale). In contrast, our approach keeps attention time constant across the full 3K-frame generation. **(b) Peak memory.** Peak GPU memory for FA3 (and other baselines) grows with context length due to KV-cache expansion, whereas our method stays essentially flat throughout generation, indicating a constant-size KV cache. Together, these results confirm that our approach avoids both latency and memory growth for long-horizon world-model generation.

Table 3 further reports results on MAGI-1 and SkyReels-V2, following *exactly the same evaluation protocol* as (Anonymous, 2026). Across both models, TempCache-Quant (ours) achieves the best efficiency–quality trade-off: it reduces compute (PFLOPs) and latency while maintaining or improving generation quality. On MAGI-1, TempCache-Quant lowers compute to 110 PFLOPs and improves speed to $4.11\times$ with higher VBench (78.99%) and strong perceptual fidelity (LPIPS 0.3156, SSIM 0.6555, PSNR 23.12). On SkyReels-V2, it further scales to $9.25\times$ speedup with the lowest compute (11 PFLOPs) and latency (146s), while preserving near-baseline VBench (83.82% vs. 83.84%). These results demonstrate that TempCache provides substantial acceleration without compromising visual quality, outperforming prior caching baselines under a fair, identical setup.

Table 4 reports **5-second** generation results on HunyuanVideo and Wan2.1-14B. Across both models, **AnnSA-LSH/Quant (ours)** achieves quality comparable to (and in several metrics slightly better than) existing sparse-attention baselines, while

Table 4. Results on **5-second** video generation for HunyuanVideo (117 frames) and Wan2.1-14B (69 frames). AnnSA (LSH/Quant) matches or slightly improves the quality of prior sparse-attention baselines while staying in a similar efficiency regime. On these short clips, the lowest latency / highest speedup is still achieved by STA (FA3), since sparse-attention kernels introduce extra overhead and are typically less optimized than FlashAttention at short sequence lengths. Our main benefits appear in long-horizon generation where attention and KV-cache growth dominate compute and memory.

Model	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Vision Reward $\uparrow$	PFLOPs $\downarrow$	Latency (s) $\downarrow$	Speedup $\uparrow$
HunyuanVideo (117 frames)	Original	–	–	–	0.141	612	1649	–
	STA (FA3)	26.7	0.866	0.167	0.132	331	719	2.29$\times$
	PA	22.1	0.764	0.256	0.140	339	1002	1.65 $\times$
	SVG	27.2	0.895	0.114	0.144	340	867	1.90 $\times$
	RadianAttn	27.3	0.886	0.114	0.139	339	876	1.88 $\times$
	AnnSA-LSH (ours)	27.2	0.899	0.112	0.149	322	899	1.77 $\times$
	AnnSA-Quant (ours)	27.4	0.900	0.111	0.150	341	912	1.81 $\times$
Wan2.1-14B (69 frames)	Original	–	–	–	0.136	560	1630	–
	STA (FA3)	22.9	0.830	0.171	0.132	322	812	2.01$\times$
	PA	22.4	0.790	0.176	0.126	324	978	1.67 $\times$
	SVG	23.2	0.825	0.202	0.114	324	949	1.71 $\times$
	RadianAttn	23.9	0.842	0.163	0.128	323	917	1.77 $\times$
	AnnSA-LSH (ours)	23.5	0.844	0.161	0.131	354	987	1.71 $\times$
	AnnSA-Quant (ours)	27.4	0.839	0.159	0.133	366	912	1.66 $\times$

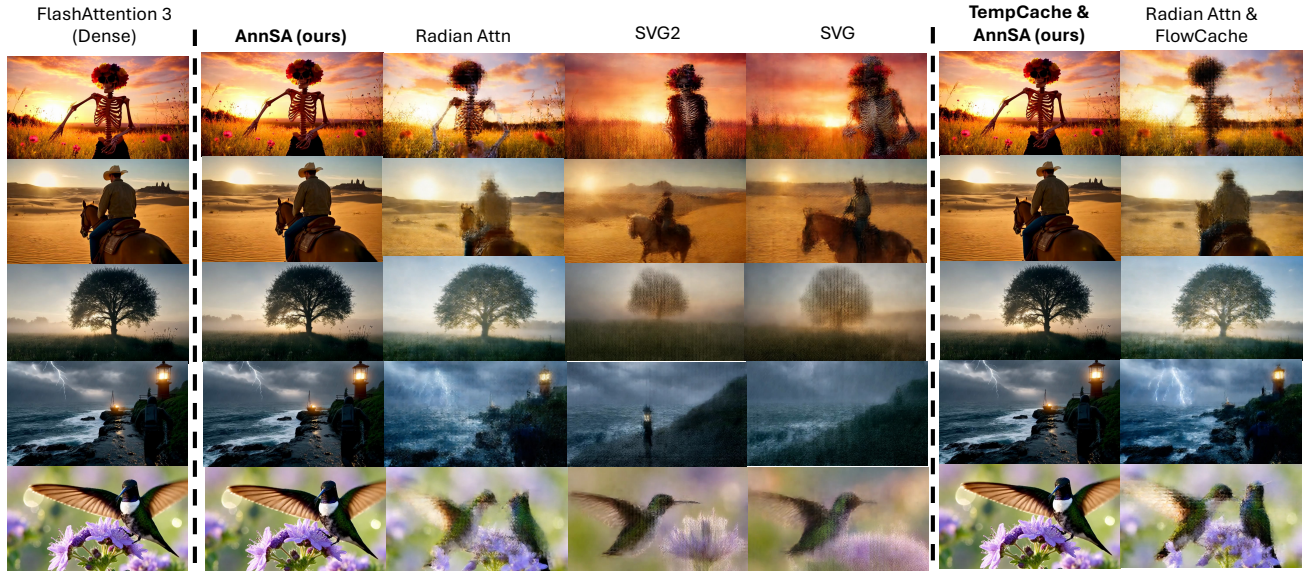


Figure 9. **Qualitative results on video world-model LongVie2.** Our TempCache+ANN sparsification preserves the visual fidelity and temporal consistency of dense FlashAttention-3 across diverse prompts, while offline-designed sparsification baselines (SVG1/2) often exhibit artifacts and drift; RadialAttention is more stable but still degrades in challenging scenes. full videos and additional results can be found in the supplementary material.

remaining in the same efficiency regime. Notably, the best latency/speedup is still attained by **STA (FA3)** on these short clips, since sparse attention relies on kernels that incur additional overhead and are typically less optimized than FlashAttention at short sequence lengths. These results indicate that our method is already competitive for short videos, while our main gains emerge in long-horizon generation where attention and KV-cache growth dominate runtime and memory (cf. long-context experiments).

Table 5. **DEVIL benchmark with motion-intensity categorization.** We evaluate the LSH variant of our method using Rolling Forcing on medium- and high-dynamic prompts. All components accelerate generation relative to dense FlashAttention-3. Speedups are lower for high-dynamic prompts, where temporal redundancy is reduced, but quality remains close to the dense baseline due to the dense-attention fallback mechanism.

Prompt Type	Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	VBench $\uparrow$	Min Density $\downarrow$	Max Recall $\uparrow$	Total Speedup $\uparrow$
Medium-Dynamic Prompts	Dense (FA3)	–	–	–	82.45	100%	100%	$\times 1.0$
	TempCache (ours)	24.12	0.651	0.147	82.52	21.8%	90.4%	$\times 6.4$
	AnnSA (ours)	25.41	0.674	0.135	82.27	35.0%	91.8%	$\times 4.9$
	AnnCA (ours)	24.85	0.662	0.142	82.41	31.2%	92.5%	$\times 2.1$
	All Ours	25.05	0.661	0.139	82.44	–	–	$\times 9.7$
High-Dynamic Prompts	Dense (FA3)	–	–	–	79.82	100%	100%	$\times 1.0$
	TempCache (ours)	22.94	0.625	0.171	79.74	39.8%	86.7%	$\times 3.9$
	AnnSA (ours)	24.28	0.644	0.158	79.55	49.1%	89.1%	$\times 3.2$
	AnnCA (ours)	23.67	0.631	0.166	79.68	46.5%	90.3%	$\times 1.9$
	All Ours	23.81	0.633	0.165	79.78	–	–	$\times 5.8$

B.2. Evaluation on Motion-Intensity Categorized Prompts

To further evaluate our method under explicitly categorized motion regimes, we additionally test on the DEVIL benchmark, which provides prompts grouped by motion intensity. We evaluate the LSH variant of our method using Rolling Forcing on medium- and high-dynamic prompts. As shown in Table 5, all proposed sparsification components continue to provide acceleration in these more challenging settings. The gains are somewhat smaller for high-dynamic prompts, as expected, since fast motion reduces temporal redundancy and weakens attention sparsity. Nevertheless, our method remains faster than dense FlashAttention-3 while preserving comparable VBench scores. Importantly, when the attention pattern becomes insufficiently sparse, our method falls back to dense attention, preventing quality degradation in highly dynamic cases.

B.3. Qualitative Results

Figure 9 shows additional qualitative comparisons on **LongVie2** rollouts. Our method remains visually close to dense FlashAttention-3, preserving scene layout, subject identity, and lighting over time, even in challenging cases with thin structures and low-contrast textures. In contrast, offline-designed sparsification baselines (SVG1/2) often accumulate artifacts during long-horizon generation, leading to blur, texture wash-out, and partial disappearance or distortion of the main subject. RadialAttention is generally more stable than SVG-style sparsity but still exhibits noticeable degradation in several examples. Overall, these LongVie2 results further support that our KV compression together with ANN-based SA/CA sparsity maintains temporal coherence and perceptual quality under long rollouts while enabling efficient generation.

B.4. Human Preference Study

We additionally conduct a blind pairwise human preference study to compare the perceptual fidelity of our accelerated generations against prior baselines. We generate 50 videos using our method and competing methods, and ask raters to choose which video in each pair more closely matches the corresponding video generated with dense FlashAttention-3. This protocol directly measures whether the approximation introduced by sparse attention and KV compression is perceptually closer to the dense-attention reference.

Table 6 reports the results. AnnSA is preferred over RadialAttn in 89% of pairwise comparisons and over SVG2 in 93% of comparisons. The full method, All Ours, is preferred over the strongest baseline combination FlowCache+RadialAttn in 92% of comparisons. Overall, our approach is preferred in 91.3% of all pairwise judgments, supporting the quantitative results and indicating that the proposed ANN-based sparsification preserves perceptual quality more faithfully than prior sparse-attention baselines.

Table 6. **Blind pairwise human preference study.** Raters compare videos generated by our method and baselines, selecting which one more closely matches the dense FlashAttention-3 reference. Our method is preferred in 91.3% of pairwise judgments overall.

Comparison	Preferred Method	Preference Rate
AnnSA (ours) vs. RadialAttn	AnnSA (ours)	89%
AnnSA (ours) vs. SVG2	AnnSA (ours)	93%
All Ours vs. FlowCache+RadialAttn	All Ours	92%
Overall	Ours	91.3%

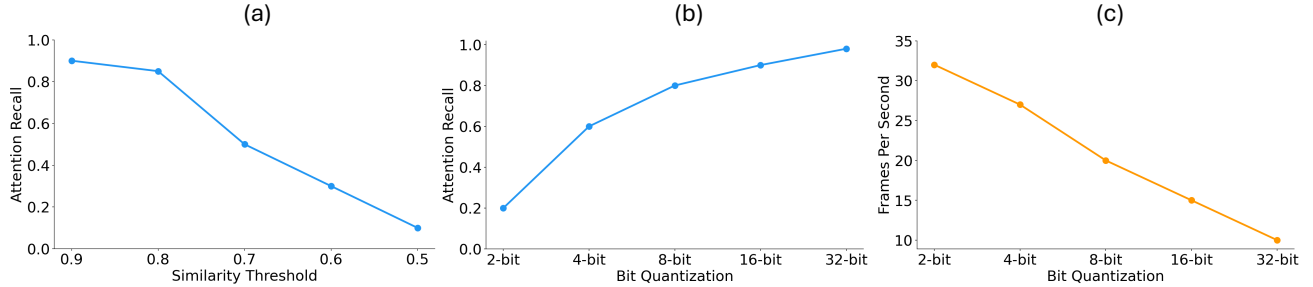


Figure 10. **Ablations on TempCache and quantized ANN.** (a) TempCache KV compression trades accuracy for compression: lowering the key-similarity threshold increases merging but reduces attention recall. (b) Quantization bit-width vs. recall: higher precision improves ANN matching quality. (c) Quantization bit-width vs. throughput: lower precision yields higher FPS, highlighting the accuracy-speed trade-off.

C. Ablation Study

TempCache similarity threshold. Figure 10(a) analyzes the key design trade-offs in TempCache. TempCache merges temporally corresponding keys when their similarity exceeds a threshold. As the threshold decreases from 0.9 to 0.5, the compression becomes more aggressive, but attention recall drops from 0.90 to 0.10, indicating that over-merging can remove keys that still contribute meaningfully to attention. This motivates using conservative thresholds that capture near-duplicate temporal features while avoiding excessive approximation.

Quantization precision for ANN matching. Figure 10 analyzes the key design trade-offs our quantization-based ANN matching. We vary the bit-width used for ANN retrieval. Increasing precision improves matching quality, raising recall from 0.20 (2-bit) to 0.98 (32-bit), while also reducing throughput as computation and memory bandwidth increase (FPS decreases from 32 to 10 from 2-bit to 32-bit). Overall, we observe a clear accuracy-efficiency trade-off, where mid-precision settings (e.g., 8-bit) provide a strong balance, achieving high recall (≈ 0.80) with substantial speed gains.

Representative selection in KV merging. When grouping temporally corresponding keys, TempCache must choose a single representative per group. We compare (i) *Last-key* (ours), which keeps the most recent key, (ii) *Mean-key*, which averages keys in the group, and (iii) *Medoid-key*, which selects the most central key. Last-key performs best, achieving 90% attention recall versus 75% for mean-key and 66% for medoid-key. This aligns with autoregressive generation: current-frame queries are most compatible with recent context, whereas averaging or selecting a central past key can blur recency-specific features and distort the attention distribution.

Warmup ratio. We ablate the warmup ratio used before enabling ANN-based sparsification. During early diffusion steps, attention patterns are less sparse and ANN matching can introduce overhead without yielding substantial pruning. Conversely, delaying sparsification for too long misses useful acceleration opportunities in later steps. As shown in Table 7, generation quality remains nearly unchanged across warmup ratios, with VBench staying around 84.0. This stability is due to our dense-attention fallback, which avoids applying sparse attention when the detected sparsity is insufficient. The best speed is obtained with a 30% warmup ratio, which provides the best balance between avoiding early overhead and exploiting sparsity in later denoising steps.

Table 7. Ablation on warmup ratio. Quality remains stable across settings because our method falls back to dense attention when sparsity is insufficient. A 30% warmup ratio gives the best efficiency by avoiding early ANN overhead while still exploiting sparsity in later steps.

Warmup Ratio	VBench $\uparrow$	Observation
Low warmup	~ 84.0	Early ANN overhead; limited useful sparsity
30%	~ 84.0	Best speed–quality trade-off
High warmup	~ 84.0	Misses useful sparsification opportunities

Table 8. Ablation on block sparsity threshold. Quality remains stable across thresholds, while speed is maximized at an 85% threshold. Lower thresholds sparsify blocks that are not sufficiently sparse, whereas higher thresholds are too conservative.

Block Sparsity Threshold	VBench $\uparrow$	Observation
Lower threshold	~ 84.0	Sparsifies weakly sparse blocks; less efficient
85%	~ 84.0	Best speed–quality trade-off
Higher threshold	~ 84.0	Too conservative; misses useful sparsification

Block sparsity threshold. We also ablate the block sparsity threshold that determines when a block is sparse enough to benefit from sparse attention. As summarized in Table 8, quality is stable across thresholds, again due to the dense-attention fallback. However, efficiency depends strongly on this threshold. Lower thresholds are too aggressive and apply sparse attention to blocks that are not truly sparse, reducing the benefit due to overhead and imperfect pruning. Higher thresholds are overly conservative and miss many blocks where sparsification would be beneficial. We find that an 85% threshold gives the best speed while maintaining stable quality.